



RUGGINE CHAT

Manuale del Progettista

Documentazione tecnica dell'architettura e dell'implementazione

Rust · Tokio · Axum · WebSocket · SQLite · egui

Indice

1. Descrizione del progetto e requisiti
2. Architettura generale del sistema
3. Struttura del workspace Cargo
4. Crate ruggine_common — Tipi condivisi
5. Crate ruggine_server — Server WebSocket e HTTP
6. Crate ruggine_client — Client GUI
7. Protocollo di comunicazione WebSocket
8. Persistenza dati — SQLite
9. Monitoraggio CPU e sistema di logging
10. Ottimizzazioni build e dimensione eseguibile
11. Concorrenza e gestione degli errori
12. Dipendenze del progetto
13. Piattaforme supportate e compilazione

1. Descrizione del progetto e requisiti

Ruggine è un sistema client/server per la gestione di una chat di scambio di messaggi testuali in tempo reale, sviluppato interamente nel linguaggio Rust. Il nome del progetto è un gioco di parole sulla traduzione italiana del termine "Rust" (ruggine).

1.1 Requisiti funzionali soddisfatti

- Registrazione al primo avvio: il client invia una richiesta Register al server che crea il profilo nel database SQLite
- Login automatico: se lo username esiste già, l'utente viene riconnesso senza password
- Gruppi di chat privati: creazione, gestione e accesso esclusivamente su invito
- Sistema di inviti completo: InviteUser, AcceptInvite, RejectInvite con stato persistente
- Comunicazione in tempo reale via WebSocket su porta 4000
- Interfaccia grafica nativa cross-platform tramite egui/eframe

1.2 Requisiti non funzionali soddisfatti

- Disponibilità su più piattaforme: Windows 10/11, Linux (Ubuntu 22.04+), macOS
- Logging CPU ogni 120 secondi nel file cpu.log tramite la libreria sysinfo
- Ottimizzazione della dimensione degli eseguibili tramite profilo release dedicato
- Basso consumo di CPU a regime (< 1% misurato nei log reali)

2. Architettura generale del sistema

L'architettura di Ruggine segue il modello client/server classico, con comunicazione bidirezionale in tempo reale tramite WebSocket. Il server espone anche una API HTTP REST per operazioni ausiliarie.

2.1 Schema dei componenti

Componente	Responsabilità
<code>ruggine_server</code>	Gestione connessioni WebSocket, HTTP API REST, persistenza SQLite, routing messaggi, monitoraggio CPU
<code>ruggine_client</code>	Interfaccia grafica nativa (egui/eframe), comunicazione asincrona con il server, rendering messaggi e gestione comandi utente
<code>ruggine_common</code>	Definizione dei tipi condivisi ClientMessage, ServerMessage, UserDTO, GroupInfo con serializzazione <code>serde_json</code>

2.2 Flusso di comunicazione

Il client e il server comunicano in modo full-duplex tramite WebSocket. Il client GUI gira sul thread principale (requisito di egui/eframe), mentre il runtime Tokio è eseguito in un thread secondario dedicato. I due thread comunicano tramite canali mpsc non bloccanti.

Schema del flusso interno al client:



3. Struttura del workspace Cargo

Il progetto è organizzato come workspace Cargo con tre crate indipendenti che condividono il file Cargo.lock e il profilo release.

```
ruggine/
├── Cargo.toml          # Workspace root + profilo release
├── Cargo.lock          # Lock delle dipendenze
├── ruggine.db          # Database SQLite (generato a runtime)
├── cpu.log             # Log monitoraggio CPU (generato a runtime)
├── ruggine_server/
│   ├── Cargo.toml
│   └── src/
│       ├── main.rs     # Entry point, TCP listener WebSocket
│       ├── state.rs    # AppState condiviso
│       └── http/
│           ├── mod.rs  # Router Axum + CORS
│           ├── handlers.rs # Handler HTTP (health, users, login)
│           └── routes.rs # Definizione rotte
├── ruggine_client/
│   ├── Cargo.toml
│   └── src/
│       └── main.rs     # App egui + websocket_task
└── ruggine_common/
    ├── Cargo.toml
    └── src/
        └── lib.rs      # ClientMessage, ServerMessage, UserDTO, GroupInfo
```

4. Crate ruggine_common — Tipi condivisi

Il crate `ruggine_common` funge da libreria di tipi condivisi tra server e client. Non contiene logica applicativa ma solo definizioni di strutture dati serializzabili.

4.1 Enum ClientMessage

Messaggi inviati dal client al server tramite WebSocket, serializzati come JSON:

Variante	Campi	Descrizione
Register	username: String	Registrazione o login al primo avvio
SendMessage	group_id: i64, content: String	Invia un messaggio a un gruppo
CreateGroup	name: String	Crea un nuovo gruppo
InviteUser	group: String, username: String	Invita un utente nel gruppo per nome
AcceptInvite	group: String	Accetta un invito pendente
RejectInvite	group: String	Rifiuta un invito pendente
LeaveGroup	group: String	Lascia un gruppo di cui si è membri
DirectMessage	to_username: String, content: String	Messaggio diretto a un singolo utente
ListGroups	(none)	Richiede la lista dei gruppi di cui si è membri

4.2 Enum ServerMessage

Messaggi inviati dal server al client come risposta o notifica:

Variante	Campi	Descrizione
RegisterResponse	success: bool, reason: Option<String>	Esito della registrazione/login
NewMessage	group_id: i64, sender_username: String, content: String, timestamp: i64	Nuovo messaggio ricevuto da un gruppo
InviteReceived	group_id: i64, group_name: String, inviter_username: String	Notifica di invito ricevuto
GroupCreated	id: i64, name: String	Conferma creazione gruppo
JoinedGroup	group_id: i64, group_name: String	Conferma ingresso in un gruppo
UserJoinedGroup	group_id: i64, username: String	Notifica che un utente si è unito
InviteSent	group: String, username: String	Conferma che l'invito è stato inviato

InviteRejected	group: String	L'utente ha rifiutato l'invito
GroupList	groups: Vec<GroupInfo>	Lista dei gruppi dell'utente
LeftGroup	group_id: i64, group_name: String	L'utente ha lasciato il gruppo
UserLeftGroup	group_id: i64, group_name: String, username: String	Notifica agli altri membri che un utente ha lasciato il gruppo
DirectMessageReceived	from_username: String, content: String, timestamp: i64	Messaggio diretto ricevuto da un altro utente
DirectMessageResult	to_username: String, success: bool, reason: Option<String>,	Conferma che il DM è stato consegnato
Error { message }		Errore generico o messaggio di sistema

5. Crate ruggine_server — Server WebSocket e HTTP

5.1 Entry point (main.rs)

Il main del server svolge le seguenti operazioni in sequenza:

1. Inizializza il sistema di logging strutturato tramite tracing-subscriber con output su file rotante in logs/
2. Apre o crea il database SQLite ruggine.db tramite rusqlite
3. Esegue le migrazioni DDL (CREATE TABLE IF NOT EXISTS) per users, groups, group_members, pending_invites
4. Crea lo AppState condiviso (Arc<AppState>) con il database e la mappa degli utenti attivi
5. Avvia il task di monitoraggio CPU in background (ogni 120 secondi)
6. Avvia il server HTTP Axum su 0.0.0.0:3000 in un task Tokio separato
7. Avvia il listener WebSocket su 0.0.0.0:4000, accettando connessioni concorrenti

5.2 Stato condiviso (state.rs)

Lo stato condiviso è definito come:

```
pub struct AppState {
    pub db: Mutex<Connection>,           // Database SQLite
    pub active_users: Mutex<HashMap<String, ClientTx>>, // Canali TX attivi
}
pub type SharedState = Arc<AppState>;
```

La struttura è avvolta in Arc per la condivisione tra i task Tokio. I Mutex garantiscono l'accesso esclusivo al database e alla mappa degli utenti connessi. ClientTx è un alias per mpsc::UnboundedSender<ServerMessage>.

5.3 HTTP API (http/)

Il modulo http/ gestisce le rotte REST tramite Axum. Il router viene costruito con layer CORS permissivo (allow_origin: Any) per facilitare lo sviluppo.

Metodo	Endpoint	Handler	Descrizione
GET	/health	health_check()	Health check — risponde "OK"
GET	/users	get_users()	Restituisce lista di UserDTO (id, username)
POST	/login	login()	Verifica esistenza utente, login senza password

5.4 Gestione connessioni WebSocket

Ogni client che si connette al server tramite WebSocket ottiene un task dedicato. Il server:

- Crea un canale mpsc per inviare messaggi al client (ClientTx)
- Registra il client nella mappa active_users all'avvio
- Gestisce il messaggio Register per creare/recuperare l'utente nel database
- Instrada i messaggi SendMessage ai soli membri del gruppo destinatario

- Gestisce LeaveGroup con DELETE su group_members, inviando LeftGroup al client uscente e UserLeftGroup ai membri rimanenti in active_users
- Gestisce DirectMessage con SELECT su users per verificare il destinatario, consegna tramite active_users e risponde con DirectMessageResult al mittente
- Rimuove il client dalla mappa active_users alla disconnessione

6. Crate ruggine_client — Client GUI

6.1 Architettura interna del client

Il client separa nettamente la logica UI dalla logica di rete tramite due thread e canali mpsc:

```
main()
├─ std::thread::spawn(||
tokio::runtime::Runtime::block_on(websocket_task(...)))
└─ eframe::run_native("Ruggine Chat", ..., RuggineApp::new(...))
```

Questa separazione è obbligata dal vincolo di egui/eframe che richiede il thread principale per il loop UI, mentre Tokio necessita del proprio executor per le operazioni I/O asincrone.

6.2 Struttura RuggineApp

La struttura RuggineApp implementa il trait eframe::App con il metodo update() chiamato ad ogni frame. I campi principali sono:

Campo	Tipo	Uso
ui_to_ws_tx	mpsc::UnboundedSender<ClientMessage>	Invia comandi al WebSocket task
ws_to_ui_rx	mpsc::UnboundedReceiver<ServerMessage>	Riceve risposte dal server
messages	Vec<ChatMessage>	Buffer locale dei messaggi visualizzati
groups	Vec<GroupInfo>	Lista gruppi di cui si è membri
active_group	Option<GroupInfo>	Gruppo attualmente selezionato
is_registered	bool	Flag per mostrare login vs chat UI

6.3 Loop di rendering

Il metodo update() viene chiamato ogni frame (~50 ms grazie a ctx.request_repaint_after). Ad ogni frame:

8. Drena il canale ws_to_ui_rx tramite try_recv() non bloccante (process_server_messages)
9. Renderizza la schermata di login se !is_registered, altrimenti la UI principale
10. Il pannello sinistro lista i gruppi; fare clic imposta active_group
11. L'area centrale mostra solo i messaggi con group_id == active_group.id

6.4 Gestione comandi

I comandi vengono intercettati in handle_command() quando l'input inizia con /. Non comportano logica locale complessa: vengono convertiti in varianti ClientMessage e inviati al server tramite il canale.

7. Protocollo di comunicazione WebSocket

Il protocollo utilizza messaggi JSON su WebSocket (testo, non binario). La serializzazione è gestita automaticamente da `serde_json` tramite i tag `#[derive(Serialize, Deserialize)]` sulle enum.

7.1 Formato messaggi

Le enum Rust con varianti a campi nominati vengono serializzate da `serde` come oggetti JSON con un campo "tipo":

```
{ "Register": { "username": "mario" } }
{ "SendMessage": { "group_id": 1, "content": "Ciao!" } }
{ "NewMessage": { "group_id": 1, "sender_username": "mario",
                  "content": "Ciao!", "timestamp": 1736259600 } }
```

7.2 Sequenza tipica di una sessione

12. Client si connette a `ws://127.0.0.1:4000`
13. Client invia `Register { username }`
14. Server risponde `RegisterResponse { success: true, reason: None }`
15. Client invia `ListGroups`
16. Server risponde `GroupList { groups: [...] }`
17. Client invia `CreateGroup { name: "generale" }`
18. Server risponde `GroupCreated { id: 1, name: "generale" }`
19. Client invia `SendMessage { group_id: 1, content: "Ciao!" }`
20. Server instrada `NewMessage` a tutti i membri del gruppo

8. Persistenza dati — SQLite

Il server utilizza SQLite embedded tramite rusqlite con la feature bundled (libreria C compilata staticamente nel binario, nessuna dipendenza di sistema).

8.1 Schema del database

```
CREATE TABLE IF NOT EXISTS users (  
    id          INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT UNIQUE NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS groups (  
    id    INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT UNIQUE NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS group_members (  
    group_id INTEGER REFERENCES groups(id),  
    user_id  INTEGER REFERENCES users(id),  
    PRIMARY KEY (group_id, user_id)  
);
```

```
CREATE TABLE IF NOT EXISTS pending_invites (  
    group_name TEXT NOT NULL,  
    username   TEXT NOT NULL,  
    PRIMARY KEY (group_name, username)  
);
```

8.2 Considerazioni

- Le query utilizzano prepared statements per prevenire SQL injection
- Il Mutex<Connection> garantisce accesso serializzato al database da più task Tokio
- Il file ruggine.db viene creato automaticamente al primo avvio
- La feature bundled di rusqlite compila SQLite staticamente, eliminando dipendenze runtime

9. Monitoraggio CPU e sistema di logging

9.1 Monitoraggio CPU

Il server avvia un task Tokio dedicato al monitoraggio periodico del consumo CPU del processo. Il task:

21. Si registra come task di background con `tokio::spawn`
22. Utilizza la libreria `sysinfo` per ottenere le statistiche di processo
23. Legge il PID corrente tramite `std::process::id()`
24. Attende 120 secondi con `tokio::time::sleep`
25. Scrive una riga nel file `cpu.log` nel formato: `[YYYY-MM-DD HH:MM:SS] PID: <pid> - CPU usage: <pct>%`

Dati reali raccolti dal file `cpu.log` del progetto:

Timestamp	PID	CPU Usage	Note
2026-01-07 14:24:15	7540	92.47%	Primo campione — inizializzazione runtime
2026-01-07 14:26:15	7540	1.71%	A regime — idle
2026-01-07 14:28:15	7540	0.86%	A regime — messaggi attivi
2026-01-07 14:54:15	7540	6.50%	Picco breve — nuova connessione client
2026-02-20 10:50:38	21196	29.02%	Avvio nuovo processo
2026-02-20 10:52:38	21196	10.95%	Stabilizzazione
2026-02-20 11:01:54	23904	6.85%	Sessione di test con più client

9.2 Sistema di log applicativo

Il logging applicativo è gestito tramite il crate tracing con `tracing-subscriber`. I log vengono scritti su file rotante nella directory `logs/` con prefisso `ruggine_server.log`. Il livello di default è `INFO`. Le macro `info!()` e `warn!()` sono usate nei handler HTTP per tracciare le operazioni principali.

10. Ottimizzazioni build e dimensione eseguibile

Il file Cargo.toml del workspace contiene un profilo release specificamente configurato per minimizzare la dimensione degli eseguibili:

```
[profile.release]
opt-level = "z"      # Ottimizza per la dimensione minima
lto = true           # Link-Time Optimization: rimuove codice morto
codegen-units = 1    # Singola unità: massima ottimizzazione LTO
panic = "abort"      # Elimina il codice di unwinding dello stack
strip = true         # Rimuove simboli debug dal binario
```

10.1 Effetti delle ottimizzazioni

Impostazione	Effetto sulla dimensione	Impatto sulle prestazioni
opt-level = "z"	Riduzione significativa (~30-40% vs opt-level=3)	Leggera riduzione velocità vs opt=3
lto = true	Rimozione codice non raggiunto tra crate	Compile time più lungo
codegen-units = 1	Migliore LTO su tutto il codebase	Compile time molto più lungo
panic = "abort"	Elimina il runtime di unwinding (~100 KB)	Panic non recuperabile
strip = true	Rimuove tutti i simboli debug	Debugging impossibile senza DWARF

10.2 Dimensione degli eseguibili

Con il profilo release attivo, le dimensioni tipiche degli eseguibili sono:

Eseguibile	Dimensione approssimativa	Piattaforma
ruggine_server	~2–4 MB	Linux x86-64
ruggine_server.exe	~3–5 MB	Windows x86-64
ruggine_client	~3–5 MB	Linux x86-64
ruggine_client.exe	~4–6 MB	Windows x86-64

NOTA

Il client ha dimensione superiore al server perché include la libreria grafica egui/eframe con i font embedded. La feature bundled di rusqlite aggiunge circa 1 MB al server per SQLite compilato staticamente.

11. Concorrenza e gestione degli errori

11.1 Modello di concorrenza

Il server Ruggine utilizza il modello di concorrenza asincrona di Tokio (`async/await`). Ogni connessione WebSocket viene gestita da un task Tokio indipendente, permettendo migliaia di connessioni simultanee con un consumo di memoria molto inferiore rispetto ai thread OS tradizionali.

- `Arc<AppState>`: lo stato condiviso è avvolto in `Arc` per il passaggio sicuro tra task
- `Mutex<Connection>`: l'accesso al database è serializzato tramite `Mutex` di `std`
- `mpsc::UnboundedSender`: la comunicazione server→client è non bloccante
- `tokio::select!`: il client WebSocket usa `select!` per gestire in parallelo invio e ricezione

11.2 Gestione degli errori

Il codice usa `Result` e `?` per la propagazione degli errori. I handler HTTP ritornano `Result<Json<T>, (StatusCode, String)>`. Il main del server usa `anyhow::Result` per errori di avvio fatali. Non vengono usati `unwrap()` nel percorso critico — gli errori vengono gestiti con `match` o propagati.

SCELTA TECNICA

L'uso di `Mutex<Connection>` invece di un pool asincrono (come `sqlx`) è una scelta consapevole di semplicità: il database SQLite è locale e le query sono veloci, quindi la contesa sul `Mutex` è trascurabile nel contesto di una chat.

12. Dipendenze del progetto

12.1 ruggine_server

Crate	Versione	Utilizzo
axum	0.7	Framework HTTP async — routing e handler REST
tokio	1.35 (full)	Runtime async — task, timer, I/O asincrono
tokio-tungstenite	0.21	Server WebSocket asincrono su Tokio
rusqlite	0.31 (bundled)	SQLite embedded — persistenza utenti, gruppi, inviti
sysinfo	0.30	Statistiche di processo — utilizzo CPU per cpu.log
tracing / tracing-subscriber	0.1 / 0.3	Logging strutturato con output su file rotante
tracing-appender	0.2	File appender non bloccante per i log
tower-http	0.5 (cors)	Middleware CORS per l'API HTTP
serde / serde_json	1.0	Serializzazione/deserializzazione JSON
chrono	0.4	Timestamp per cpu.log
anyhow	1.0	Gestione errori ergonomica nel main
futures-util	0.3	Stream e Sink utilities per WebSocket

12.2 ruggine_client

Crate	Versione	Utilizzo
eframe	0.27	Framework app nativa cross-platform (wrappa egui)
egui	0.27	Libreria GUI immediate-mode — layout e widgets
tokio-tungstenite	0.21	Client WebSocket asincrono
tokio	1 (full)	Runtime async nel thread separato
serde / serde_json	1.0	Serializzazione messaggi WebSocket
anyhow	1.0	Gestione errori
futures-util	0.3	SinkExt / StreamExt per WebSocket

12.3 ruggine_common

Crate	Versione	Utilizzo
serde	1.0 (derive)	Macro Serialize/Deserialize per le enum di protocollo
serde_json	1.0	Serializzazione JSON del protocollo WebSocket

13. Piattaforme supportate e compilazione

13.1 Piattaforme testate

Piattaforma	Stato	Note
Windows 10/11 (x86-64)	Testato	MSVC toolchain consigliata; binario .exe standalone
Linux Ubuntu 22.04+ (x86-64)	Testato	Richiede librerie grafiche (libgtk su alcuni DE)
macOS 12+ (Intel/ARM)	Supportato	Richiede Xcode Command Line Tools

13.2 Requisiti di compilazione

- Rust 1.75 o superiore (edition 2024)
- Cargo (incluso con rustup)
- Su Linux: pkg-config e librerie di sistema per egui (xorg-dev o wayland-dev)
- Su macOS: Xcode Command Line Tools (xcode-select --install)
- Su Windows: Visual Studio Build Tools o MSVC

13.3 Comandi di build

```
# Build debug (rapida, con simboli)
cargo build
```

```
# Build release (ottimizzata per produzione)
cargo build --release
```

```
# Build solo server
cargo build --release -p ruggine_server
```

```
# Build solo client
cargo build --release -p ruggine_client
```

```
# Verifica dimensione binari (Linux/macOS)
ls -lh target/release/ruggine_server target/release/ruggine_client
```

13.4 Cross-compilazione

Rust supporta la cross-compilazione tramite cargo e rustup target add. Per compilare il server Linux su Windows:

```
rustup target add x86_64-unknown-linux-gnu
cargo build --release --target x86_64-unknown-linux-gnu -p ruggine_server
```

NOTA CROSS-C OMPIL.

La feature bundled di rusqlite semplifica la cross-compilazione del server evitando la necessità di linkare SQLite di sistema sulla piattaforma target.